Day 27 – Middle of Linked List

🚀 Middle of Linked List

💡 Problem

Linked List diya hai → middle node find karna hai

👉 Example:

Input: 1 → 2 → 3 → 4 → 5
Output: 3 (middle)

Input: 1 → 2 → 3 → 4 → 5 → 6
Output: 4 (second middle)

🧠 Approach (Best Method – Slow & Fast Pointer)

👉 Ye sabse important aur interview favorite approach hai
👉 Isse kehte hain Two Pointer Technique

⚙️ Logic:

Ek pointer slow (1 step)

Ek pointer fast (2 step)

👉 Jab fast end tak pahunchta hai
👉 tab slow middle par hota hai

📊 Visualization

Example:

1 → 2 → 3 → 4 → 5

Step	Slow	Fast
Start	1	1
1	2	3
2	3	5

End	3	null

👉 Answer = 3

🔁 Even Case Dry Run

1 → 2 → 3 → 4 → 5 → 6

Step	Slow	Fast
Start	1	1
1	2	3
2	3	5
3	4	null

👉 Answer = 4 (second middle)

✅ Java Code

```java
package Day_27;

public class MiddleOfLinkedList {

    class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }

    Node head;

    // Find Middle
    public Node findMiddle() {
        if (head == null) return null;

        Node slow = head;
        Node fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;        // move 1 step
            fast = fast.next.next;   // move 2 steps
        }

        return slow;
    }

    // Print Linked List
    public void printList() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        MiddleOfLinkedList list = new MiddleOfLinkedList();

        list.head = list.new Node(1);
        list.head.next = list.new Node(2);
        list.head.next.next = list.new Node(3);
        list.head.next.next.next = list.new Node(4);
        list.head.next.next.next.next = list.new Node(5);

        list.printList();

        Node middle = list.findMiddle();
        System.out.println("Middle Node: " + middle.data);
    }
}

```

⏱️ Complexity

Time: O(n)

Space: O(1) (No extra space)